Here are the exam-ready answers and programs for **Internal Test 3** (Part A and the first half of Part B):

PART - A (2-Mark Questions)

Question 1:

> **What are STL containers? Name any two sequence containers and any two associative containers.** *(2 Marks)*

- **Answer:**
  - **STL Containers:** Data structures stored as template classes in the Standard Template Library that manage collections of objects (vectors, lists, maps, etc.).
  - **Sequence Containers:** `std::vector`, `std::list`
  - **Associative Containers:** `std::map`, `std::set`

Question 2:

> **What is the difference between `map` and `multimap` in STL? Give an example.** *(2 Marks)*

- **Answer:**
  - `std::map`**:** Stores unique key-value pairs. Each key can appear only once.
  - `std::multimap`**:** Allows multiple elements to have the exact same key (duplicate keys are allowed).
  - **Example:** A phonebook where names must be unique is a `map`, but a database mapping a teacher's name to multiple student names is a `multimap`.

Question 3:

> **Write a C++ program to demonstrate a race condition.** *(2 Marks)*

- **Answer:**

```cpp
#include <iostream>
#include <thread>
using namespace std;
int counter = 0;
void increment() { for(int i=0; i<10000; i++) counter++; }
int main() {
    thread t1(increment), t2(increment);
    t1.join(); t2.join();
    cout << counter; // Unpredictable output due to race condition
    return 0;
}
```

Question 4:

> **Write a C++ program to iterate through a `map<int, string>` and display its key-value pairs.** *(2 Marks)*

- **Answer:**

```cpp
#include <iostream>
#include <map>
using namespace std;
int main() {
    map<int, string> m = {{1, "One"}, {2, "Two"}};
    for (auto it = m.begin(); it != m.end(); ++it)
        cout << it->first << ": " << it->second << endl;
    return 0;
}
```

Question 5:

> **Write a C++ program to demonstrate `fork()` system call.** *(2 Marks)*

- **Answer:**

```cpp
#include <iostream>
#include <unistd.h>
using namespace std;
int main() {
    pid_t pid = fork();
    if (pid == 0) cout << "Child Process\n";
    else cout << "Parent Process\n";
    return 0;
}
```

PART - B (First Half)

Question 6.a.i:

> **Write a program to traverse a vector `<int>` using an iterator.** *(2 Marks)*

- **Answer:**

```cpp
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> v = {10, 20, 30};
    for (auto it = v.begin(); it != v.end(); ++it)
        cout << *it << " ";
```

```
    return 0;
}
```

Question 6.a.ii:

> **A `std::thread` object `t` is joinable. What does calling `t.join()` do?** *a) Allows t to run independently by detaching it. b) Pauses the thread represented by t until another thread resumes it. c) Blocks the calling thread until the thread represented by t finishes. d) Immediately terminates the thread represented by t. (2 Marks)*

- **Correct Answer: c) Blocks the calling thread until the thread represented by t finishes.**
- **Justification:** `join()` suspends execution of the calling thread (usually `main`) until the worker thread completes its task, preventing the program from exiting prematurely.

Question 6.b.ii: Two threads increment the same ordinary int counter. Which approach prevents a data race during these increments? a) Call join() on both threads after starting them. b) Protect every increment using std::lock_guard with the same mutex. c) Declare counter as volatile. d) Call sleep_for() before each increment. (2 Marks) Correct Answer: b) Protect every increment using std::lock_guard with the same mutex. Justification: std::lock_guard together with a std::mutex provides mutual exclusion, ensuring that only one thread can modify the shared counter at any given time, preventing race conditions.